

Học Sâu (Deep Learning)

Chương 9: Các Mẫu Kiến trúc Mạng Tích chập (ConvNet Architecture Patterns)

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Nguyên lý Thiết kế Kiến trúc
2. Kết nối Thặng dư (Residual Connections)
3. Batch Normalization (Chuẩn hóa Hàng loạt)
4. Tích chập Tách rời (Separable Convolution)
5. Thực hành: Xây dựng Mini-Xception
6. Kỹ nguyên mới: Vision Transformers (ViTs)
7. Tổng kết

Không gian Giả thuyết (Hypothesis Space)

- **Kiến trúc Mô hình (Model Architecture)** là nghệ thuật lắp ráp các tầng mạng (Layers).
- Cấu trúc này định nghĩa **Không gian Giả thuyết** (Hypothesis Space). Thuật toán Gradient Descent sẽ tìm kiếm bộ tham số (Weights) tối ưu bên trong không gian này.
- Một kiến trúc tốt giống như một bản thiết kế thông minh, giúp đóng gói **Tri thức tiên nghiệm (Prior Knowledge)** của kỹ sư vào mô hình.
- Ví dụ: Việc sử dụng Tích chập (Convolution) thay cho mạng Dense hàm chứa tri thức tiên nghiệm rằng: *"Các mẫu cục bộ trong hình ảnh là bất biến với sự dịch chuyển"*.

Tầm quan trọng của Thiết kế Kiến trúc

- Thiết kế khéo léo sẽ làm **thu hẹp không gian tìm kiếm**, giúp Gradient Descent hội tụ nhanh hơn về điểm tối ưu.
- Nó cũng giúp mô hình sử dụng dữ liệu hiệu quả hơn, chống lại hiện tượng Overfitting ngay cả khi tập dữ liệu huấn luyện khá nhỏ.
- Ngược lại, nếu chọn một kiến trúc sai lầm (ví dụ: dùng mạng Dense để nhận diện ảnh 4K), mô hình sẽ mãi mắc kẹt ở hiệu suất thấp, và không có một lượng dữ liệu lớn nào có thể cứu vãn được.

Nghệ thuật Thiết kế vs. Bằng chứng Khoa học

- Thiết kế kiến trúc thường nghiêng về **Nghệ thuật (Art)** hơn là Khoa học chính xác (Science).
- Các kỹ sư máy học xuất chúng thường sử dụng **Trực giác (Intuition)** và kinh nghiệm nhận dạng mẫu (Pattern Matching) để ghép nối mô hình, thay vì dựa vào các chứng minh toán học khắt khe.
- Sự thật: Không ai có thể giải thích bằng toán học một cách hoàn hảo *tại sao* một kiến trúc lại hoạt động tốt hơn kiến trúc khác. Ta chỉ biết nó hiệu quả thông qua thực nghiệm.

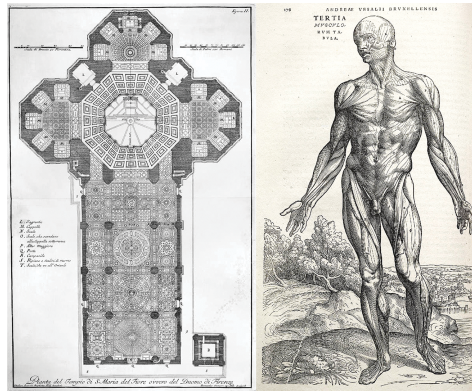
Ablation Studies (Nghiên cứu Cắt bỏ)

- Trong bối cảnh ngành Deep Learning có nhiều cấu trúc phức tạp được công bố chỉ để làm bài báo trông "nguy hiểm" hơn (Tối ưu hóa cho Peer Review).
- Mục tiêu thực sự của kỹ sư là **Tìm hiểu quan hệ Nhân quả**.
- **Ablation Studies:** Kỹ thuật gỡ bỏ từng thành phần của mô hình để kiểm chứng xem sự phức tạp được thêm vào có thực sự mang lại hiệu quả không.
- *Ví dụ:* Nếu mô hình $X + Y + Z$ hoạt động tốt, hãy thử chạy lại với chỉ X , Y , Z , $X+Y$... để biết chính xác sức mạnh đến từ đâu.

Công thức MHR: Modularity, Hierarchy, Reuse

Để quản lý một hệ thống phức tạp, ta dùng nguyên lý **MHR**:

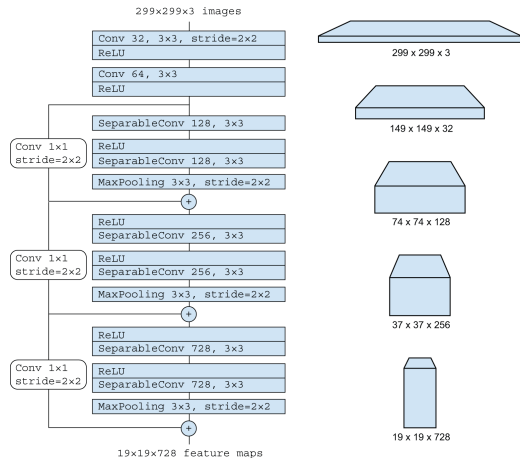
- **Modularity (Mô-đun hóa)**: Đóng gói các lớp tính toán cơ bản thành các Khối (Blocks) độc lập.
- **Hierarchy (Phân cấp)**: Xếp chồng các khối thành hệ thống hình Tháp (Pyramid).
- **Reuse (Tái sử dụng)**: Tái sử dụng cùng một cấu trúc khối lặp đi lặp lại.



Hình 9.1: Hệ thống phức tạp sinh học tuân theo nguyên lý MHR

Tháp Đặc trưng trong Mô hình Xception

- Các mạng CNN hiện đại (như Xception, ResNet) tuân thủ chặt chẽ nguyên lý MHR.
- Cấu trúc Lặp lại (Reuse): Khối $\text{SeparableConv} \rightarrow \text{SeparableConv} \rightarrow \text{MaxPooling}$.
- **Tháp không gian:** Kích thước bản đồ giảm dần (Shrinking: $299 \times 299 \rightarrow 19 \times 19$).
- **Tháp chiều sâu:** Số lượng bộ lọc (Filters) tăng dần (Deepening: $32 \rightarrow 728$).



Hình 9.2: Sự phân cấp của Khối đầu vào Xception

Vấn đề: Suy biến Đạo hàm (Vanishing Gradient)

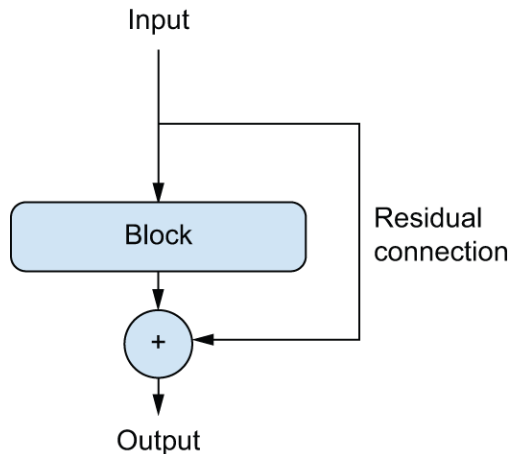
- Mạng càng **Sâu (Deep)**, hệ thống phân cấp (Hierarchy) càng hiệu quả vì nó ép mô hình tạo ra các đặc trưng trừu tượng cao cấp (Abstraction).
- Trực giác mạch bảo: Cứ xếp chồng thêm lớp, mô hình sẽ càng thông minh.
- Tuy nhiên, khi xếp chồng quá nhiều lớp (ví dụ: hàng trăm lớp Conv2D), mạng đột ngột **ngừng học**, Loss không suy giảm.
- Hiện tượng này không phải do Overfitting, mà do **Vanishing Gradient**.

Quy tắc Chuỗi (Chain Rule) và Trò chơi Truyền tin

- Thuật toán Backpropagation truyền đạo hàm ngược từ hàm Loss về các tầng đầu thông qua **Quy tắc chuỗi (Chain Rule)**.
- Quá trình này giống như trò chơi *Truyền tin tĩnh điện (Chinese Whispers/Telephone)*: Thông điệp qua từng người sẽ bị nhiễu dần.
- Phép nhân liên tiếp nhiều số nhỏ hơn 1 (qua các hàm ReLU, lớp Dropout) sẽ khiến đạo hàm giảm mạnh theo cấp số nhân về số 0.
- Các tầng đầu tiên không nhận được bất kỳ tín hiệu cập nhật nào, mô hình bị "tê liệt".

Giải pháp: Kết nối Thặng dư (Residual Connections)

- Phát minh của Kaiming He (2015) trong kiến trúc **ResNet** (Đạt giải Best Paper tại CVPR).
- Cung cấp một **đường tắt (Shortcut)** cho thông tin nhảy vọt qua khối tính toán (không bị biến đổi).
- Đầu vào được cộng trực tiếp vào đầu ra:
 $y = F(x) + x$.
- Đạo hàm chảy ngược xuyên qua các "đường cao tốc" này về đầu mạng một cách mượt mà.



Hình 9.3: Cơ chế hoạt động của Khối Residual

Mã nguồn giả (Pseudocode) cho Residual

Thay vì bắt khối (block) học ra một kết quả ánh xạ $F(x)$ hoàn toàn mới (rất khó), ta chỉ bắt nó học phần **thặng dư (Residual)** còn thiếu so với đầu vào.

```
1 # Some input tensor
2 x = ...
3 # Saves a reference to the original input. This is called the residual.
4 residual = x
5
6 # This computation block can potentially be destructive or noisy.
7 # It only needs to learn the "difference" (residual).
8 x = block(x)
9
10 # Adds the original input to the layer's output. The final output will
11 # thus always preserve full information about the original input.
12 x = add([x, residual])
```

Mã nguồn: Cân bằng số lượng Bộ lọc (Filters)

Phép cộng `add([x, residual])` đòi hỏi 2 Tensor phải có cùng hình dáng (Shape). Khi Khối Convolution làm tăng số lượng Channels, ta phải dùng một lớp Conv2D 1×1 để chiếu (Project) đường dẫn thẳng dư lên cùng số Channels.

```
1 inputs = keras.Input(shape=(32, 32, 3))
2 x = layers.Conv2D(32, 3, activation="relu")(inputs)
3 residual = x
4
5 # Block increases output filters from 32 to 64
6 x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
7
8 # Project the residual using 1x1 Conv2D to match 64 filters
9 residual = layers.Conv2D(64, 1)(residual)
10
11 x = layers.add([x, residual])
```

Mã nguồn: Cân bằng Kích thước (MaxPooling)

Tương tự, khi không gian bị thu nhỏ bởi MaxPooling2D, ta cũng phải thu nhỏ đường dẫn Residual bằng tham số `strides=2` để bảo đảm khớp kích thước khi cộng.

```
1 inputs = keras.Input(shape=(32, 32, 3))
2 x = layers.Conv2D(32, 3, activation="relu")(inputs)
3 residual = x
4
5 # Block includes MaxPooling downsampling
6 x = layers.Conv2D(64, 3, activation="relu", padding="same")(x)
7 x = layers.MaxPooling2D(2, padding="same")(x)
8
9 # Project the residual and apply downsampling with strides=2
10 residual = layers.Conv2D(64, 1, strides=2)(residual)
11
12 x = layers.add([x, residual])
```

Tại sao cần Chuẩn hóa Kích hoạt (Activation Normalization)?

- Chuẩn hóa dữ liệu đầu vào (căn giữa Mean=0, tỷ lệ Variance=1) giúp mô hình dễ hội tụ. (Ví dụ: scale pixel từ 0-255 về 0-1).
- Nhưng khi dữ liệu đi sâu vào các lớp bên trong, sự phân bố liên tục bị biến đổi sau mỗi phép nhân ma trận.
- Các tầng phía sau liên tục phải thích nghi với dữ liệu đầu vào mới thay đổi liên tục. Sự bất ổn này gọi là **Internal Covariate Shift**.
- **Batch Normalization** (Ioffe & Szegedy, 2015) giải quyết vấn đề bằng cách *chuẩn hóa lại dữ liệu ngay trong lúc đang di chuyển* giữa các tầng.

Toán học của Batch Normalization

Lớp BatchNormalization chuẩn hóa từng lô (Batch) trong lúc Training:

- 1 Tính Kỳ vọng (μ) của Batch: $\mu = \frac{1}{m} \sum_{i=1}^m x_i$
- 2 Tính Phương sai (σ^2) của Batch: $\sigma^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu)^2$
- 3 Chuẩn hóa gốc: $\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$
- 4 **Dịch chuyển và Tỷ lệ (Scale & Shift):** $y_i = \gamma \hat{x}_i + \beta$

Trong đó, γ và β là 2 tham số **được học** qua Gradient Descent. Nó cho phép mô hình *hủy bỏ* tác dụng của BN và khôi phục phân phối gốc nếu việc chuẩn hóa làm mất đi thông tin quý giá.

Quá trình Suy luận (Inference Mode)

- Khi Đưa vào sử dụng (Inference/Predict), ta thường dự đoán trên từng ảnh đơn lẻ (Batch Size = 1). Không thể tính μ và σ^2 của 1 ảnh được!
- Giải pháp: Trong suốt quá trình Training, lớp BN âm thầm cập nhật một **Trung bình động lũy thừa (Exponential Moving Average)** của các μ và σ^2 từ tất cả các batches.
- Khi Inference, nó dùng hằng số Moving Average này để chuẩn hóa thay vì tính toán lại.
- *Lưu ý khi Transfer Learning*: Luôn đóng băng (Freeze) lớp BN bằng 'trainable=False' để tránh làm hỏng Moving Average đã được học rất tốt từ ImageNet.

Cách lập trình Batch Normalization chuẩn xác

Vì BN tự động chuẩn hóa Mean về 0, tham số bias của tầng Convolution trước đó trở nên vô dụng (bị hấp thụ bởi β). Ta nên tắt nó bằng `use_bias=False` để tiết kiệm RAM.

Cách KHÔNG NÊN dùng (Activation bị đặt sai chỗ):

```
1 x = layers.Conv2D(32, 3, activation="relu")(x)
2 x = layers.BatchNormalization()(x)
```

Cách SỬ DỤNG CHUẨN (BN phải đứng TRƯỚC Hàm kích hoạt):

```
1 # Note the lack of bias and activation here
2 x = layers.Conv2D(32, 3, use_bias=False)(x)
3 x = layers.BatchNormalization()(x)
4 # We place the activation AFTER the BatchNormalization layer
5 x = layers.Activation("relu")(x)
```

Giới hạn của Tích chập Truyền thống (Conv2D)

- Lớp Conv2D thực hiện đồng thời 2 việc trong 1 bước tính toán duy nhất:
 - ① Học các mẫu Không gian 2D (Spatial Pattern - VD: đường viền).
 - ② Tổ hợp thông tin chéo từ nhiều Kênh (Cross-channel Mixing - VD: Kết hợp kênh Đỏ và Xanh).
- Chi phí tính toán cực kỳ tốn kém, số tham số: $K^2 \times C_{in} \times C_{out}$.
- **Giả thuyết mới:** Đặc trưng Không gian và Đặc trưng Kênh có tính độc lập cao. Tại sao không tách chúng ra?

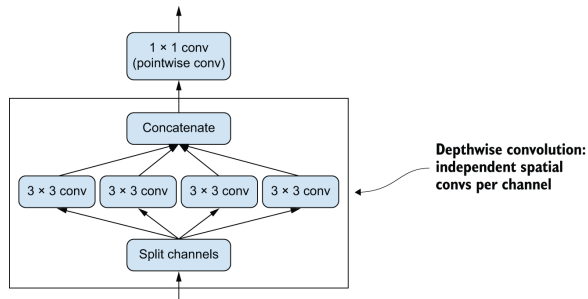
Tích chập Tách rời (Depthwise Separable Convolution)

Tách phép tính thành 2 pha độc lập
(SeparableConv2D):

- 1 **Depthwise Conv:** Quét không gian trên từng Kênh một cách độc lập (Không trộn lẫn).
- 2 **Pointwise Conv (1×1):** Nhào trộn thông tin giữa các Kênh tại từng điểm ảnh.

Số lượng tham số giảm mạnh:

$$K^2 \times C_{in} + C_{in} \times C_{out}.$$



Hình 9.4: Tích chập Tách rời chiều sâu

- Giả sử ta dùng Kernel 3×3 , đầu vào có 64 Channels, đầu ra có 64 Channels.
- **Tích chập chuẩn (Conv2D):**
 - Số tham số $= 3^2 \times 64 \times 64 = 36,864$ tham số.
- **Tích chập Tách rời (SeparableConv2D):**
 - Số tham số $= (3^2 \times 64) + (64 \times 64) = 576 + 4,096 = 4,672$ tham số.
- **Kết luận:** Tiết kiệm gần 90% bộ nhớ và năng lực tính toán. Mô hình hội tụ nhanh hơn, chống Overfitting tốt hơn do ít tham số dư thừa.

Sự co-evolve (Tiến hóa đồng thời) của Phần cứng và Phần mềm

- Dù tiết kiệm 90% tham số, nhưng trên GPU thực tế, SeparableConv2D chỉ nhanh **tương đương** Conv2D thông thường. Tại sao?
- Bởi vì **NVIDIA cuDNN** đã tối ưu vi mô (micro-optimized) kernel Conv2D tiêu chuẩn bằng hợp ngữ (Assembly) qua hàng thập kỷ cho hàng Exaflops tính toán.
- Các thuật toán mới dù ưu việt về toán học nhưng không được phần cứng ưu tiên thiết kế → Sẽ chạy rất chậm.
- Đây là rào cản khổng lồ (Sự tiến hóa đồng thời của Hardware - Software - Algorithm) ngăn chặn các đột phá AI đi chệch khỏi lối mòn của Backpropagation.

Lắp ráp mô hình Mini-Xception (Phần 1)

Chương trình sau ứng dụng cả 3 bảo bối: **SeparableConv**, **BatchNormalization**, và **Residual Connections**.

```
1 inputs = keras.Input(shape=(180, 180, 3))
2 x = layers.Rescaling(1.0 / 255)(inputs)
3
4 # First layer must be regular Conv2D because RGB channels
5 # are highly correlated in natural images. Separable assumption fails here!
6 x = layers.Conv2D(filters=32, kernel_size=5, use_bias=False)(x)
7
8 for size in [32, 64, 128, 256, 512]:
9     residual = x
10
11     x = layers.BatchNormalization()(x)
12     x = layers.Activation("relu")(x)
13     x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
```

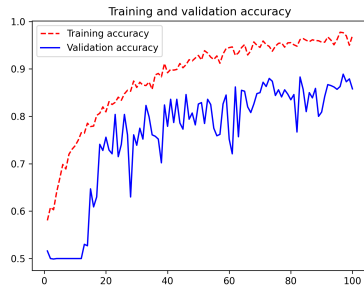
Lắp ráp mô hình Mini-Xception (Phần 2)

Kết thúc vòng lặp, ta gắn khối Classifier (Toàn cục). Kích thước mạng cực nhẹ, chỉ tốn 721,857 tham số so với 1.5 triệu tham số của CNN cũ!

```
1 x = layers.BatchNormalization()(x)
2 x = layers.Activation("relu")(x)
3 x = layers.SeparableConv2D(size, 3, padding="same", use_bias=False)(x)
4 x = layers.MaxPooling2D(3, strides=2, padding="same")(x)
5
6 # Project residual block
7 residual = layers.Conv2D(size, 1, strides=2, padding="same", use_bias=False)(
8     residual)
9 x = layers.add([x, residual])
10
11 x = layers.GlobalAveragePooling2D()(x)
12 x = layers.Dropout(0.5)(x)
13 outputs = layers.Dense(1, activation="sigmoid")(x)
14 model = keras.Model(inputs=inputs, outputs=outputs)
```


Đánh giá Độ chính xác (Accuracy)

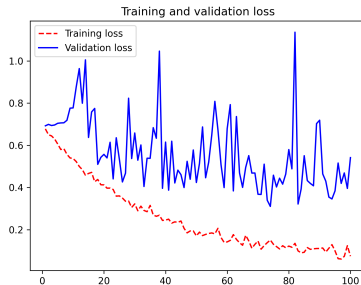
- Nhờ cấu trúc siêu ưu việt, mô hình tự thiết kế của chúng ta đạt **Accuracy lên đến 90.8%** trên tập Chó/Mèo.
- Mô hình Basic CNN ở Chương 8 (Không dùng Transfer Learning) chỉ đạt được mốc 83.9%.
- Thiết kế kiến trúc chuẩn mực (Architecture Best Practices) mang lại **Bước nhảy vọt** về hiệu năng!



Hình 9.5: Accuracy của Mini-Xception

Đánh giá Mất mát (Loss)

- Đường Validation Loss đi xuống ổn định và mượt mà hơn.
- Sự hiện diện của BatchNormalization ở mọi ngõ ngách giúp thuật toán hội tụ cực nhanh.
- Dạng kiến trúc lặp (Xception) hiện là tiêu chuẩn công nghiệp (Industry Standard) trên Keras.



Hình 9.6: Loss của Mini-Xception

Giao thời của Công nghệ (Beyond Convolution)

- Mạng Tích chập (ConvNets) đã độc tôn thống trị lĩnh vực Thị giác Máy tính suốt từ thập niên 2010.
- Gần đây, một Kiến trúc hoàn toàn mới đang đe dọa thay thế ngôi vương này: **Vision Transformers (ViTs)**.
- Transformers vốn dĩ được sinh ra vào năm 2017 dành riêng cho Xử lý Ngôn ngữ Tự nhiên (NLP - để dịch thuật văn bản).
- Ý tưởng táo bạo: Chuyện gì sẽ xảy ra nếu ta coi một bức ảnh giống như là "Một câu văn", và các bộ phận của ảnh là "Các từ vựng"?

- Mạng ViT hoàn toàn **bác bỏ** toán tử Conv2D để quét ảnh.
- Nó chặt bức ảnh gốc thành một lưới các mảnh ghép (Patches) có kích thước cố định, ví dụ 16×16 pixel.
- Trải phẳng mỗi mảnh ghép thành một Vector 1 chiều (Biến nó thành một "Từ vựng" - Token).
- Bơm toàn bộ chuỗi Vector vào kiến trúc Transformer để dùng cơ chế **Self-Attention** tìm kiếm mối tương quan.

Sức mạnh của Self-Attention

- Mạng ConvNet gặp khó khăn trong việc kết nối các chi tiết ở xa nhau trên ảnh do kích thước Kernel (3×3) quá nhỏ bé. Phải xếp lớp rất sâu mới nhìn được toàn cảnh.
- Mạng ViT với cơ chế Self-Attention có khả năng so sánh **mọi Patch với mọi Patch khác** cùng một lúc.
- Điều này tạo ra khả năng liên kết cự ly xa (Long-range relationships) tuyệt vời. Nó hiểu ngay lập tức quả bóng ở góc trái có liên quan mật thiết đến con chó đang chạy ở góc phải bức ảnh.

Hạn chế của ViTs so với ConvNets

Tuy rất mạnh, ViTs có những điểm yếu chí mạng đối với các dự án thực tế:

- **Cần dữ liệu khổng lồ:** ConvNets có lợi thế "Kiến thức ưu tiên 2D" (Biết rõ các pixel cạnh nhau thì liên quan đến nhau). ViTs là một tờ giấy trắng, nó phải tự học lại định luật vật lý về không gian → Phải đào tạo trên lượng dữ liệu khổng lồ (Lớn hơn cả ImageNet).
- **Mô hình quá đồ sộ:** Mạng ViT cực nặng, đòi hỏi hạ tầng Siêu máy tính (Clusters of TPUs/GPUs) khổng lồ để huấn luyện.
- Do đó, với các bài toán doanh nghiệp thông thường và dữ liệu có giới hạn, **ConvNets vẫn là sự lựa chọn hoàn hảo nhất.**

Tổng kết Chương 9

Để xây dựng một mô hình CNN xuất sắc từ con số 0, hãy tuân theo các nguyên tắc sau:

- Tổ chức Mô hình theo dạng Tháp Đặc trưng (Feature Pyramid). Lặp lại Khối (Blocks) thay vì tự nối từng Layer ngẫu nhiên (Quy tắc MHR).
- Sử dụng **Separable Convolution** thay cho Convolution tiêu chuẩn (Tiết kiệm 90% năng lực tính toán).
- Kết hợp **Batch Normalization** ngay trước Hàm Kích hoạt để hệ thống hội tụ mượt mà, triệt tiêu Covariate Shift.
- Tạo các **Kết nối Thặng dư (Residual Connections)** để dòng chảy Gradient truyền xa xuyên suốt mạng sâu hàng trăm tầng mà không bị suy biến.